

Implementation Plan

Hospital Bed Allocation System

Version: 1.0
Date: 2026-05-17
Duration: 4 Weeks (College Project Timeline)
Team Size: 1-2 Students
Methodology: Waterfall with incremental DB-first development

1. Project Overview

1.1 Objective

Build a complete Hospital Bed Allocation System that demonstrates mastery of DBMS concepts (normalization, triggers, stored procedures, cursors, views, transactions, constraints) through a functional Java Swing desktop application.

1.2 Success Metrics

- All 8 tables created in 3NF/BCNF with proper constraints
- 6+ triggers enforcing business rules
- 4+ stored procedures including cursor-based operations
- 6+ views for reporting
- Functional Swing GUI with 7+ screens
- ACID-compliant transactions for admission/discharge
- Working waitlist with auto-allocation
- Complete project report with ER diagram, normalization trace, and screenshots

2. Phase Breakdown

Phase 1: Database Design & Setup (Days 1-5)

Goal: Complete, tested database schema with all advanced objects

Day	Task	Deliverable	DBMS Concepts Demonstrated
1	Requirements analysis, ER diagram design	ER Diagram (Crow's Foot notation)	Entity-Relationship modeling
1	Normalization trace document	UNF → 1NF → 2NF → 3NF → BCNF document with FDs	Normalization, Functional Dependencies
2	Write schema.sql —all CREATE TABLE statements	schema.sql file	Data types, constraints, keys

Table 1 – continued

Day	Task	Deliverable	DBMS Concepts Demonstrated
2	Add CHECK, FOREIGN KEY, UNIQUE, NOT NULL constraints	Updated <code>schema.sql</code>	Domain integrity, referential integrity
3	Write <code>triggers.sql</code> –all 6 triggers	<code>triggers.sql</code> file	Triggers, assertions (via triggers)
3	Write <code>procedures.sql</code> – <code>sp_allocate_bed</code> , <code>sp_discharge_patient</code> , <code>sp_process_waitlist</code> , <code>sp_update_waitlist_priority</code>	<code>procedures.sql</code> file	Stored procedures, cursors, transactions
4	Write <code>views.sql</code> –all 6 views	<code>views.sql</code> file	Views, complex JOINS, aggregations
4	Write <code>indexes.sql</code> – performance indexes	<code>indexes.sql</code> file	Indexing, query optimization
5	Write <code>seed_data.sql</code> – sample data for testing	<code>seed_data.sql</code> file	Data population
5	Test all SQL scripts in MySQL Workbench	Tested database	Verification

Phase 1 Checklist:

- Run `schema.sql` without errors
- Verify all foreign keys work (try inserting invalid FK → should fail)
- Verify CHECK constraints (try invalid status → should fail)
- Test triggers: insert admission → bed status auto-updates
- Test triggers: try double-booking a bed → should fail with error message
- Test procedures: call `sp_allocate_bed` with valid data → returns `bed_id`
- Test procedures: call `sp_allocate_bed` when no beds → adds to waitlist
- Test procedures: call `sp_discharge_patient` → bed frees, waitlist auto-allocates
- Test cursors: `sp_process_waitlist` allocates multiple patients
- Test views: all views return correct data
- Verify indexes exist: `SHOW INDEX FROM beds;`

Phase 2: Java Backend (DAO Layer) (Days 6–10)

Goal: Complete data access layer with JDBC connectivity

Day	Task	Deliverable	Concepts
6	Set up Maven project structure, add MySQL JDBC dependency	pom.xml, project skeleton	Build tools
6	Create DBConnection.java — Singleton pattern, connection pooling basics	DBConnection.java	JDBC, Singleton pattern
7	Create Model/POJO classes: Ward.java, Bed.java, Patient.java, Staff.java, Admission.java, Waitlist.java	model/ package	OOP, encapsulation
7	Create WardDAO.java — CRUD operations	WardDAO.java	JDBC CRUD
8	Create BedDAO.java — findAvailableBed, updateStatus, getStatistics	BedDAO.java	JDBC queries, ResultSet handling
8	Create PatientDAO.java —insert, search, getById	PatientDAO.java	JDBC, search patterns
9	Create AdmissionDAO.java — allocateBed (calls stored proc), dischargePatient (calls stored proc), getActiveAdmissions	AdmissionDAO.java	CallableStatement, transactions
9	Create WaitlistDAO.java —getWaitlist, updatePriority, removeFromWaitlist	WaitlistDAO.java	JDBC updates
10	Create StaffDAO.java, ReportDAO.java (calls views)	StaffDAO.java, ReportDAO.java	View consumption
10	Write JUnit tests for DAO layer (optional but recommended)	Test classes	Unit testing

Key Implementation Details:

DBConnection.java

```
package com.hospital.db;

import java.sql.*;
```

```

public class DBConnection {
    private static final String URL = "jdbc:mysql://localhost:3306/hospital_bed_db?
    ↪ useSSL=false&serverTimezone=UTC";
    private static final String USER = "root";
    private static final String PASSWORD = "your_password";
    private static Connection connection;

    public static Connection getConnection() throws SQLException {
        if (connection == null || connection.isClosed()) {
            connection = DriverManager.getConnection(URL, USER, PASSWORD);
        }
        return connection;
    }

    public static void closeConnection() {
        if (connection != null) {
            try { connection.close(); } catch (SQLException e) { e.printStackTrace(); }
        }
    }

    public static void beginTransaction() throws SQLException {
        getConnection().setAutoCommit(false);
        getConnection().setTransactionIsolation(Connection.TRANSACTION_REPEATABLE_READ);
    }

    public static void commit() throws SQLException {
        getConnection().commit();
        getConnection().setAutoCommit(true);
    }

    public static void rollback() {
        try {
            getConnection().rollback();
            getConnection().setAutoCommit(true);
        } catch (SQLException e) { e.printStackTrace(); }
    }
}

```

Calling Stored Procedure (AdmissionDAO)

```

public AllocationResult allocateBed(int patientId, String wardType,
    int doctorId, Date expectedDischarge) {
    AllocationResult result = new AllocationResult();
    try {
        Connection conn = DBConnection.getConnection();
        CallableStatement stmt = conn.prepareCall(
            "{CALL sp_allocate_bed(?, ?, ?, ?, ?, ?, ?, ?, ?)}"
        );
        stmt.setInt(1, patientId);
        stmt.setString(2, wardType);
        stmt.setInt(3, doctorId);
    }
}

```

```

        stmt.setDate(4, expectedDischarge);
        stmt.setNull(5, Types.VARCHAR); // notes
        stmt.registerOutParameter(6, Types.INTEGER); // p_bed_id
        stmt.registerOutParameter(7, Types.INTEGER); // p_admission_id
        stmt.registerOutParameter(8, Types.BOOLEAN); // p_success
        stmt.registerOutParameter(9, Types.VARCHAR); // p_message

        stmt.execute();

        result.setBedId(stmt.getInt(6));
        result.setAdmissionId(stmt.getInt(7));
        result.setSuccess(stmt.getBoolean(8));
        result.setMessage(stmt.getString(9));

    } catch (SQLException e) {
        result.setSuccess(false);
        result.setMessage("Database error: " + e.getMessage());
    }
    return result;
}

```

Phase 2 Checklist:

- Maven build succeeds (mvn clean compile)
- DBConnection successfully connects to MySQL
- All DAO methods tested with sample data
- Stored procedures called successfully via CallableStatement
- Transaction rollback works on simulated failure
- Views queried and results mapped to Java objects

Phase 3: Swing GUI Development (Days 11–17)

Goal: Complete, functional desktop interface

Day	Task	Deliverable	UI Components
11	Create MainFrame.java — window setup, sidebar layout	MainFrame.java	JFrame, JSplitPane, JPanel
11	Create LoginFrame.java —authentication screen	LoginFrame.java	JFrame, JTextField, JPasswordField
12	Create DashboardPanel.java — stats cards, bed grid	DashboardPanel.java	JPanel, JTable, custom renderer
12	Implement BedStatusRenderer.java —color-coded cells	BedStatusRenderer.java	DefaultTableCellRenderer

Table 3 – continued

Day	Task	Deliverable	UI Components
13	Create AdmitPatientDialog.java —admission form	AdmitPatientDialog.java	JDialog, form layout, validation
13	Create DischargeDialog.java — discharge form	DischargeDialog.java	JDialog, confirmation flow
14	Create WaitlistPanel.java — waitlist table	WaitlistPanel.java	JTable, sorting, action buttons
14	Create PatientManager- Frame.java —patient CRUD	PatientManagerFrame.java	JFrame, search, forms
15	Create ReportsFrame.java — report viewer	ReportsFrame.java	JTabbedPane, JTable, export
15	Create AdminPanel.java —ward/bed/staff management	AdminPanel.java	JTabbedPane, forms, tables
16	Implement ToastNotification.java —popup notifications	ToastNotification.java	JWindow, Timer
16	Implement SwingWorker for all DB operations	Background task classes	SwingWorker, progress indication
17	Polish UI: colors, spacing, fonts, icons	Updated all GUI files	Look-and-feel customization
17	Add keyboard shortcuts, tab order, focus handling	Updated all GUI files	Accessibility

GUI Implementation Standards:

Modern Color Scheme (Applied via UIManager or custom painting)

```
// In MainFrame constructor or application init
UIManager.put("Panel.background", new Color(249, 250, 251)); // #F9FAFB
UIManager.put("Table.background", Color.WHITE);
UIManager.put("Table.selectionBackground", new Color(219, 234, 254)); // #DBEAFE
UIManager.put("Button.background", new Color(37, 99, 235)); // #2563EB
UIManager.put("Button.foreground", Color.WHITE);
```

Bed Grid Implementation

```
public class DashboardPanel extends JPanel {
    private JPanel bedGridPanel;
    private BedDAO bedDAO;
```

```

public DashboardPanel() {
    setLayout(new BorderLayout());
    setBackground(new Color(249, 250, 251));

    // Stats row
    add(createStatsPanel(), BorderLayout.NORTH);

    // Bed grid
    bedGridPanel = new JPanel(new GridLayout(0, 4, 16, 16));
    bedGridPanel.setBackground(new Color(249, 250, 251));
    bedGridPanel.setBorder(BorderFactory.createEmptyBorder(20, 20, 20, 20));

    JScrollPane scrollPane = new JScrollPane(bedGridPanel);
    scrollPane.setBorder(null);
    add(scrollPane, BorderLayout.CENTER);

    loadBeds();
}

private void loadBeds() {
    SwingWorker<List<Bed>, Void> worker = new SwingWorker<>() {
        @Override
        protected List<Bed> doInBackground() throws Exception {
            return bedDAO.getAllBedsWithWard();
        }

        @Override
        protected void done() {
            try {
                List<Bed> beds = get();
                bedGridPanel.removeAll();
                for (Bed bed : beds) {
                    bedGridPanel.add(createBedCard(bed));
                }
                bedGridPanel.revalidate();
                bedGridPanel.repaint();
            } catch (Exception e) {
                showError("Failed to load beds: " + e.getMessage());
            }
        }
    };
    worker.execute();
}

private JPanel createBedCard(Bed bed) {
    JPanel card = new JPanel(new BorderLayout());
    card.setBackground(Color.WHITE);
    card.setBorder(BorderFactory.createCompoundBorder(
        BorderFactory.createLineBorder(new Color(229, 231, 235), 1),
        BorderFactory.createEmptyBorder(16, 16, 16, 16)
    ));
}

```

```

// Top colored strip
JPanel strip = new JPanel();
strip.setPreferredSize(new Dimension(0, 4));
switch (bed.getCurrentStatus()) {
    case "Available": strip.setBackground(new Color(16, 185, 129)); break;
    case "Occupied": strip.setBackground(new Color(239, 68, 68)); break;
    case "Maintenance": strip.setBackground(new Color(245, 158, 11)); break;
}
card.add(strip, BorderLayout.NORTH);

// Content
JLabel bedNumber = new JLabel(bed.getBedNumber());
bedNumber.setFont(new Font("Segoe UI", Font.BOLD, 16));

JLabel wardLabel = new JLabel(bed.getWardType());
wardLabel.setFont(new Font("Segoe UI", Font.PLAIN, 12));
wardLabel.setForeground(new Color(107, 114, 128));

card.add(bedNumber, BorderLayout.CENTER);
card.add(wardLabel, BorderLayout.SOUTH);

// Click handler
card.addMouseListener(new MouseAdapter() {
    public void mouseClicked(MouseEvent e) {
        handleBedClick(bed);
    }
});

return card;
}
}

```

Phase 3 Checklist:

- Login screen authenticates against database
- Dashboard loads bed grid with correct color coding
- Clicking Available bed opens Admit dialog
- Clicking Occupied bed opens Discharge dialog
- Admit form validates all fields before submission
- Discharge shows confirmation dialog
- Waitlist panel displays sorted data
- Reports panel shows data from views
- Admin panel allows adding wards/beds/staff
- Toast notifications appear on success/error
- No UI freezing during DB operations (SwingWorker used)

- Window is resizable and layout adapts

Phase 4: Integration & Testing (Days 18–22)

Goal: All components work together, bugs fixed

Day	Task	Method
18	Integration testing: Full admission flow	Manual test: Login → Admit → Verify DB → Check Dashboard
18	Integration testing: Full discharge flow	Manual test: Discharge → Verify bed freed → Check waitlist auto-allocation
19	Concurrent access testing	Open two app instances, try allocating same bed simultaneously
19	Constraint violation testing	Try inserting invalid FK, invalid status, duplicate unique values
20	Performance testing	Time dashboard load, allocation query, report generation
20	Edge case testing	Empty database, 100% occupancy, maintenance during admission attempt
21	Bug fixing session	Fix all identified issues
21	Code cleanup, comments, JavaDoc	Refactoring
22	Final integration test	Run complete user journeys

Test Cases:

Test ID	Scenario	Expected Result	Pass/Fail
TC-01	Admit patient to available ICU bed	Bed status → Occupied, admission record created, dashboard updates	
TC-02	Admit patient when ICU full	Patient added to waitlist, waitlist panel updates	
TC-03	Discharge patient, no waitlist	Bed → Available, audit log entry created	
TC-04	Discharge patient, waitlist exists	Bed auto-allocated to highest priority waitlisted patient	
TC-05	Two users click same available bed	One succeeds, other gets “bed just allocated” error	
TC-06	Try to admit to maintenance bed	Error: “Bed under maintenance”	

Table 5 – continued

Test ID	Scenario	Expected Result	Pass/Fail
TC-07	Delete ward with beds	Error: “Cannot delete ward with existing beds” (FK RESTRICT)	
TC-08	Insert patient age = 200	Error: CHECK constraint violation	
TC-09	Generate occupancy report	Correct percentages displayed	
TC-10	Process waitlist batch	Multiple patients allocated, summary shown	

Phase 5: Documentation & Deployment (Days 23–26)

Goal: Complete project report and runnable deliverable

Day	Task	Deliverable
23	Write project report: Introduction, objectives	Report sections 1–2
23	Write ER diagram description, normalization trace	Report section 3 (with diagrams)
24	Document schema, triggers, procedures, views	Report section 4 (with SQL code)
24	Document Java architecture, class diagrams	Report section 5
25	Add screenshots of all GUI screens	Report section 6
25	Add test cases, results, conclusion	Report sections 7–8
26	Create README.md with setup instructions	README.md
26	Build executable JAR	HospitalBedSystem.jar
26	Final review and submission	ZIP file with code + report + SQL package

Project Report Structure:

1. Introduction & Objectives
2. Literature Review (brief)
3. System Analysis & Design
 - ER Diagram
 - Normalization (UNF $\rightarrow$ BCNF with FDs)
 - Schema Design

4. Database Implementation

- Table structures
- Constraints & Keys
- Triggers (with code)
- Stored Procedures (with code)
- Views (with code)
- Indexes

5. Application Implementation

- Architecture diagram
- Class diagram
- GUI screenshots

6. Testing

- Test cases table
- Results

7. Conclusion & Future Scope

8. References

3. File Structure

```
HospitalBedSystem/
├── pom.xml                # Maven build file
├── README.md              # Setup instructions
├── sql/
│   ├── 01_schema.sql      # CREATE TABLE statements
│   ├── 02_constraints.sql  # ALTER TABLE ADD CONSTRAINT
│   ├── 03_indexes.sql     # CREATE INDEX statements
│   ├── 04_triggers.sql    # All triggers
│   ├── 05_procedures.sql  # All stored procedures
│   ├── 06_views.sql       # All views
│   ├── 07_seed_data.sql   # Sample data
│   └── 08_test_queries.sql # Validation queries
├── src/
│   └── main/
│       ├── java/
│       │   └── com/
│       │       └── hospital/
│       │           └── bedalloc/
│       │               ├── db/
│       │               │   ├── DBConnection.java
│       │               │   └── DatabaseConfig.java
│       │               ├── model/
│       │               └── Ward.java
```


4. Risk Management

Risk	Probability	Impact	Mitigation
MySQL trigger syntax errors	Medium	High	Test each trigger individually in Workbench before adding to script
Swing GUI threading issues	High	Medium	Always use SwingWorker for DB calls; never run SQL on Event Dispatch Thread
Concurrent allocation race condition	Medium	High	Use FOR UPDATE in allocation query; test with two instances
JDBC driver not found	Low	High	Include mysql-connector-j in Maven dependencies; document in README
Database connection timeout	Low	Medium	Implement connection retry logic; show user-friendly error dialog
Scope creep (adding too many features)	High	High	Stick to requirements document; mark nice-to-haves as “future scope”

5. Daily Standup Questions (Self-Check)

Ask yourself each day:

1. What did I complete yesterday?
2. What will I work on today?
3. Are there any blockers?

6. Submission Checklist

Before submitting your project, verify:

Database Layer

- All tables created with correct constraints

- All triggers tested and working
- All stored procedures tested with various inputs
- All views return correct data
- Indexes created and explained in report
- Seed data inserted successfully
- Normalization document complete (UNF $\rightarrow$ BCNF)
- ER diagram included

Application Layer

- Code compiles without errors
- All GUI screens functional
- Admission flow works end-to-end
- Discharge flow works end-to-end
- Waitlist auto-allocation tested
- Concurrent access handled safely
- Input validation on all forms
- Error handling (connection lost, invalid data)

Documentation

- Project report complete with all sections
- Screenshots of all GUI screens
- SQL scripts organized and commented
- README with setup instructions
- Executable JAR created
- All files organized in ZIP

End of Implementation Plan